

note

GROWTH · PROJECT BETA

GrowTH Backend System

Detailed Technical Description

A narrative walkthrough of backend responsibilities, end-to-end data flow for every module, and a detailed explanation of all functional requirements (FR-1 to FR-24)

Prepared for: Project Beta — Digital Media Engineering Program, Khon Kaen University

Source document: TOR — Development of the “GrowTH” Web Application (Draft for Project Kickoff)

Scope covered: TOR Section 3.3 (Backend and Database Development), Section 4 (Functional Requirements), Section 3.4, Section 6.2–6.3

Document status: Supplementary technical design note — not an official part of the TOR

Table of Contents

1. Introduction

2. Backend System Overview

3. Detailed Data Flow Description

3.1 Context-Level Flow: Parent ↔ Backend ↔ AI Service

3.2 Process 1.0 — Account and Profile Management

3.3 Process 2.0 — Growth Tracking

3.4 Process 3.0 — Puberty Development Screening

3.5 Process 4.0 — AI-Assisted Bone Age Prediction

3.6 Process 5.0 — Knowledge / Education Content

4. Detailed Functional Requirements

4.1 Account and Profile Management (FR-1 to FR-5)

4.2 Growth Tracking and Screening (FR-6 to FR-11)

4.3 Puberty Development Screening (FR-12 to FR-14)

4.4 AI-Assisted Bone Age Prediction (FR-15 to FR-19)

4.5 Knowledge / Education Section (FR-20 to FR-21)

4.6 General / Non-Functional Considerations (FR-22 to FR-24)

5. Data Store Summary

6. Summary

1. Introduction

This document provides a detailed technical description of the backend system for the GrowTH web application, as scoped in Section 3.3 (Backend and Database Development) of the Terms of Reference (TOR). It is intended as a supplementary engineering reference for the project team, expanding on the earlier ERD and Data Flow Diagram (DFD) deliverables by describing, in narrative form, exactly what data moves between the parent's browser, the backend system, the database, and the external AI inference service for every module of the application.

Two things are covered in detail below. Section 3 walks through the data flow for each backend process the same way a DFD would, but written as prose: for every process it states what triggers it, what data arrives and from where, what the backend does with that data step by step, what is written to or read from which data store, and what is sent back out and to whom. Section 4 then explains every functional requirement (FR-1 through FR-24) from the TOR in detail, from the backend's point of view — what inputs it expects, what validation and business logic it must apply, and what output it must produce.

2. Backend System Overview

The GrowTH backend sits between the responsive web front end used by parents/caregivers and two categories of persistent resources: a relational database holding all structured clinical and account data, and an object storage location holding the raw X-ray image files. The backend also acts as a client to one external system: the AI bone age inference service described in TOR Section 3.4.

At a high level, the backend is responsible for:

- Exposing a secure, documented REST API that the front end calls for every screen and action described in TOR Section 4.
- Authenticating users and maintaining sessions, including password hashing and role-appropriate access control, so a parent account can only ever reach the children, growth records, screenings, and predictions linked to that account (FR-24).
- Running all clinically meaningful calculations server-side — percentile/SDS scoring, BMI derivation, and puberty-screening result compilation — so the same logic applies consistently regardless of device or browser.
- Acting as the integration point between the front end and the AI inference microservice, so the front end never talks to the AI service directly.
- Persisting every growth entry, screening response, and bone age prediction against the correct child profile, so history and trend views are always complete and consistent.
- Enforcing data-privacy principles from TOR Section 6.2 — data minimization, secure storage of X-ray images, hashed passwords, and account-scoped access.

3. Detailed Data Flow Description

This section describes the same information as the DFDs produced earlier, but written out as text. Each subsection covers one process: what comes in, where it comes from, what happens to it inside the backend, and what goes back out and to where.

3.1 Context-Level Flow: Parent ↔ Backend ↔ AI Service

Every interaction begins in the parent's web browser. The front end sends an HTTPS request to the backend's REST API, carrying a JSON body (for data submissions) or query parameters (for reads), and — for every request after login — a session/authentication token identifying which user account is making the request.

The backend receives the request, authenticates the token, checks that the requested `child_id` (where applicable) belongs to the authenticated user, executes the relevant business logic against the database, and returns a JSON response containing either the requested data or a confirmation of the write that just occurred, plus any calculated fields such as percentile results or screening summaries.

For the bone age feature specifically, the backend additionally acts as a client: after receiving an uploaded X-ray image from the parent, it forwards that image (or a reference to it) to the AI Bone Age Inference Service over an internal API call, waits for the service to return a predicted bone age and an uncertainty figure, and then relays a formatted version of that result back to the parent — the parent's browser never communicates with the AI service directly. This mirrors the architecture described in TOR Section 3.4, which calls for a trained model packaged as an inference service/endpoint that the backend can call.

3.2 Process 1.0 — Account and Profile Management

Triggered by: Registration form submission, login form submission, or child-profile form submission on the front end.

Inputs received: For registration: `full_name`, `email`, `password`, `phone_number`, and a boolean confirming acceptance of the terms of use and privacy notice. For login: `email` and `password`. For adding a child: the child's `name`, `sex`, `date_of_birth`, and `relationship` to the account holder.

On registration, the backend first checks that the submitted email does not already exist in the `USERS` table. It then hashes the password using an industry-standard salted algorithm and stores the new row together with the `terms_accepted` flag and a `created_at` timestamp. The plain-text password is never stored.

On login, the backend looks up the `USERS` row by email and verifies the submitted password against the stored hash. If valid, it issues a session token (or signs a JWT) that the front end attaches to every subsequent request. If the credentials do not match, the backend returns a generic authentication error without revealing whether the email exists, to avoid leaking account information.

On adding a child profile, the backend takes the currently authenticated user's id from the session and inserts a new row into the `CHILDREN` table with that `user_id` as a foreign key, together with the submitted `name`, `sex`, `date_of_birth`, and `relationship`. Because every subsequent growth record, screening, and bone age prediction is keyed off `child_id`, this step is what allows the account-scoped access rule (FR-24) to be enforced everywhere else: any later request for a given `child_id` is only honoured if that child's `user_id` matches the requester's authenticated user id.

Outputs produced: A session token and the user's basic profile (on login); the newly created child record with its generated `child_id` (on adding a child); and, for the “view/switch children” screen, the full list of children linked to the account (FR-5), read back from `CHILDREN` filtered by `user_id`.

Data stores touched: Writes USERS (registration); reads USERS (login); writes CHILDREN (add child); reads CHILDREN (list/view/edit/switch children).

3.3 Process 2.0 — Growth Tracking

Triggered by: The parent submitting a new growth measurement (date, height, weight) for a selected child, or opening a child's growth history/trend view.

Inputs received: child_id, measurement_date, height_cm, weight_kg.

The backend processes a new growth entry in the following order:

- Validates the input: measurement_date must not be in the future and must not fall before the child's date_of_birth; height and weight must fall within physiologically plausible ranges. Any failure returns a clear, parent-facing error message rather than a generic server error.
- Loads the child's date_of_birth and sex from CHILDREN and computes the child's exact age in months at measurement_date.
- Looks up the matching row(s) in GROWTH_REFERENCE_DATA for that sex and age_months, retrieving the L, M, and S parameters for height-for-age and weight-for-age (and, if the computed age is 60 months or older, for BMI-for-age as well, per FR-8).
- Calculates each metric's standard deviation score (SDS/Z-score) using the LMS method, then converts each SDS value into a percentile.
- If the child is 5 years or older, computes BMI from height and weight and runs the same LMS calculation against the BMI-for-age reference curve, additionally generating a short plain-language nutritional-status summary (for example, “within normal range”).
- Evaluates whether any computed SDS value falls notably outside the expected range and, if so, generates a plain-language guidance message recommending consultation with a physician — worded as a guidance flag, not a diagnosis (FR-10).
- Persists one new row in GROWTH_RECORDS containing the raw inputs together with every calculated value, so later reads never need to repeat the calculation.

Outputs produced: The newly calculated percentile, SDS, BMI (where applicable), and guidance message are returned immediately to the front end. Whenever the parent opens the growth-history or trend-chart view, the backend returns the full ordered history of GROWTH_RECORDS for that child, which the front end plots as height-for-age, weight-for-age, and BMI-for-age line charts against standard percentile curves (FR-9, FR-11).

Data stores touched: Reads CHILDREN (age/sex); reads GROWTH_REFERENCE_DATA (LMS parameters); writes and reads GROWTH_RECORDS.

3.4 Process 3.0 — Puberty Development Screening

Triggered by: The parent completing the guided puberty-screening questionnaire for a selected child, or opening the screening history view.

Inputs received: child_id and a set of structured responses. The question set served depends on the child's sex (per Annex D: breast development/menstruation questions for female children; testicular/voice-change questions for male children) plus a shared block of general questions (family history of puberty onset, behavioural/skin changes, other relevant medical history).

Because the question set — and therefore the shape of the answer data — legitimately differs by sex and age, the backend does not force these responses into a fixed set of relational columns. It accepts the full response payload and stores it as a single structured (JSON) value, while still recording screening_date and child_id as ordinary relational columns so the record can be queried and ordered normally.

The backend then runs the compiled responses through the puberty-screening rule set (thresholds and wording to be finalized with the Client Representative, per TOR Annex D) to produce a `result_summary` and a `recommended_action`, such as suggesting consultation with a pediatrician or pediatric endocrinologist when responses suggest development outside the typical age range. The result is explicitly labelled as a screening aid, not a diagnosis, exactly as FR-13 requires.

Outputs produced: The `result_summary` and `recommended_action` are returned to the front end immediately after submission. On later visits, the backend returns the full history of past screenings for the child (FR-14) so the parent — and eventually a treating physician — can see how responses have changed across successive assessments rather than viewing a single snapshot.

Data stores touched: Writes and reads `PUBERTY_SCREENINGS`, keyed by `child_id`.

3.5 Process 4.0 — AI-Assisted Bone Age Prediction

Triggered by: The parent uploading a hand-and-wrist X-ray image for a selected child.

Inputs received: `child_id` and the image file itself, submitted as multipart/form-data.

This is the most involved backend flow because it spans three separate resources — the relational database, an object storage bucket, and the external AI service — and the steps must happen in a strict order:

- **Validate the upload:** the backend checks the file's type/extension against an accepted list and checks its size against a maximum limit, exactly as FR-16 requires. If the file fails either check, the backend responds immediately with a clear, specific error and none of the later steps run.
- **Store the image:** the validated file is written to a dedicated object storage location, kept separate from the relational database since large binary files do not belong inside relational tables, and the backend generates a unique reference (a path or object key) for it.
- **Call the AI inference service:** the backend sends the stored image (or a short-lived signed URL to it) together with the child's sex to the external bone age model's inference endpoint, as described in TOR Section 3.4, then waits for a response within a bounded, reasonable processing time (FR-17).
- **Receive the prediction:** the AI service returns the predicted bone age in months, an uncertainty/margin-of-error figure, and an identifier for the exact model version that produced the result.
- **Store the prediction:** the backend writes one new row to `BONE_AGE_PREDICTIONS` containing `child_id`, the stored image's reference (not the binary itself), `predicted_bone_age_months`, `margin_of_error`, the model version, and a `predicted_at` timestamp — satisfying the requirement in Section 3.3 that prediction records include “the uploaded image reference and the returned prediction.”
- **Compose the explanation:** the backend attaches a fixed explanatory note to the numeric result, stating in plain language what the figure means, its margin of error, and — worded exactly as FR-18 and Section 6.3 require — that it is intended to support, not replace, clinical assessment.

Outputs produced: The composed result (predicted bone age, margin of error, and disclaimer text) is returned to the front end for display alongside the child's existing growth and screening history (FR-19). On later visits, the backend returns the full history of past predictions for that child so trends can be reviewed over time.

Data stores touched: Writes to object storage (image); calls the external AI inference service; writes and reads `BONE_AGE_PREDICTIONS`.

3.6 Process 5.0 — Knowledge / Education Content

Triggered by: The parent opening or searching the knowledge/education section, independent of any specific child.

Inputs received: An optional search keyword or topic filter.

This flow does not touch any child-specific data. The backend queries the ARTICLES content store, optionally filtered by topic or keyword, and returns a list of matching entries with title and summary (FR-21). When the parent opens a specific article, the backend returns its full content. Because this content is not linked to a `child_id` or `user_id`, it carries none of the account-scoped access restrictions that apply everywhere else in the system.

Outputs produced: A browsable/searchable list of articles (FR-20) and, on request, full article content.

Data stores touched: Reads ARTICLES (a standalone content table; see the note on assumptions in Section 5).

4. Detailed Functional Requirements

This section explains every functional requirement listed in TOR Section 4, grouped by module exactly as in TOR Annex A, but expanded from the backend's point of view: what the backend must accept as input, what logic or validation it must apply, and what it must return.

4.1 Account and Profile Management (FR-1 to FR-5)

FR-1 Register a new account — The backend must accept `full_name`, `email`, `password`, and `phone_number`, enforce that email is unique, hash the password before storage, and reject incomplete submissions with field-level error messages.

FR-2 Require terms acceptance before registration completes — The backend must reject a registration request that does not include an explicit `terms_accepted` flag set to true, and must record the fact that terms were accepted (and when) against the new account.

FR-3 Secure login and logout — The backend must verify credentials against the stored password hash, issue a session token on success, and invalidate that token on logout so it can no longer be used to authenticate subsequent requests.

FR-4 Add one or more child profiles — The backend must accept `child_name`, `sex`, `date_of_birth`, and `relationship`, link the new profile to the authenticated user's `user_id`, and allow the same account to repeat this action for multiple children without limit.

FR-5 View, edit, and switch between child profiles — The backend must expose endpoints to list all children linked to the current account, update an existing child's stored fields, and retrieve a single child's data by id — always filtered by the requesting account's `user_id` so one account can never see another account's children.

4.2 Growth Tracking and Screening (FR-6 to FR-11)

FR-6 Record a growth entry — The backend must accept date of measurement, height, and weight for a specified child and persist them as a new, timestamped record rather than overwriting any previous entry.

FR-7 Calculate and display percentile and SDS for height-for-age and weight-for-age — The backend must compute both values server-side using the agreed growth reference (Section 2A.2) at the moment a growth entry is recorded, not leave this to the front end.

FR-8 Calculate BMI and BMI-for-age SDS for children aged five and above, with a plain-language nutritional summary — The backend must first check the child's computed age before deciding whether to run this calculation, since it does not apply to children under five, and must translate the numeric SDS into a short descriptive phrase such as “within normal range.”

FR-9 Display growth trends over time as line charts against standard percentile curves — The backend's role is to supply the ordered history of previously calculated values efficiently on request; the charting itself happens in the front end, but the backend must guarantee the returned series is complete and correctly ordered by date.

FR-10 Provide plain-language guidance following each growth entry, flagging results notably outside the expected range, without presenting this as a diagnosis — The backend must apply a defined threshold to

the calculated SDS values and choose appropriate wording automatically — this logic lives entirely in the backend so it is applied consistently.

FR-11 Allow the user to view a complete history of past growth entries for a child — The backend must support returning every stored GROWTH_RECORDS row for a given child_id, not just the most recent one, ordered chronologically.

4.3 Puberty Development Screening (FR-12 to FR-14)

FR-12 Provide a structured, guided questionnaire appropriate to the child's sex and age — The backend is responsible for knowing which question set to serve (or for validating that the sex-appropriate set was used) and for accepting the resulting response payload in a flexible, schema-tolerant form.

FR-13 Compile responses into a preliminary screening result, clearly labelled as a screening aid rather than a diagnosis, with guidance on next steps — The backend applies the scoring/threshold rules to the stored responses and generates both a summary and a recommended action (for example, suggesting consultation with a pediatric endocrinologist) as part of the same write operation that stores the responses.

FR-14 Allow the user to view a history of previously completed screenings — The backend must be able to return every PUBERTY_SCREENINGS row for a given child_id in chronological order, so trends in responses over time are visible.

4.4 AI-Assisted Bone Age Prediction (FR-15 to FR-19)

FR-15 Allow upload of a hand-and-wrist X-ray image for a selected child — The backend must accept a multipart file upload tied to a specific child_id and hand it off to the validation step before anything is persisted.

FR-16 Validate the uploaded file for format and size, with clear feedback on failure — The backend performs this check before writing anything to storage, so an invalid upload never reaches the object store or the AI service, and the parent receives an immediate, specific error message.

FR-17 Submit the uploaded image to the bone age prediction model and display the result within a reasonable processing time — The backend is the only component that talks to the AI inference service; it must handle the call synchronously or asynchronously as appropriate, but in either case must bound the wait and handle timeouts gracefully.

FR-18 Present the bone age result together with its margin of error and an explanatory note that it supports, but does not replace, clinical assessment — The backend must attach this explanatory text to every prediction response programmatically, rather than relying on the front end to add a disclaimer, so the wording cannot be accidentally omitted.

FR-19 Associate each bone age prediction with the corresponding child profile and growth/screening history — Every BONE_AGE_PREDICTIONS row is written with a child_id foreign key, which is what allows the front end to later display bone age results alongside that same child's growth charts and screening history.

4.5 Knowledge / Education Section (FR-20 to FR-21)

FR-20 Provide a knowledge section containing reference articles relevant to child growth and puberty development, properly attributed — The backend serves this content (from a database table or bundled

static content, pending confirmation with the Client Representative) but is responsible for ensuring no copyrighted atlas material is ever included in what it returns, per TOR Section 11.1.

FR-21 Organize content so it is searchable/browsable by topic — The backend must support filtering or querying articles by a topic field, not just returning an unstructured full list.

4.6 General / Non-Functional Considerations (FR-22 to FR-24)

FR-22 Usable on both desktop and mobile web browsers through a responsive layout — This is primarily a front-end concern, but the backend supports it by keeping API responses lightweight and consistent regardless of client device.

FR-23 Display all clinical or quasi-clinical results using plain, parent-friendly language, with underlying figures available for users who want more detail — The backend must return both the plain-language summary text and the underlying numeric fields in the same response, so the front end can choose what to surface by default versus on request.

FR-24 Ensure a user can only access the profiles and data of children linked to their own account — This is enforced at the backend on every single endpoint that takes a `child_id` parameter: before running any query, the backend confirms that the child's `user_id` matches the authenticated requester, and returns an authorization error otherwise. This single rule is what makes every other module's data safe to expose through the API.

5. Data Store Summary

For quick reference, the table below lists every data store referenced in Section 3, what it holds, and which process(es) read or write it.

Store	Holds	Written by	Read by
USERS	Account credentials and profile	1.0 (registration)	1.0 (login, profile)
CHILDREN	Child profile linked to a user	1.0 (add child)	1.0, 2.0, 3.0, 4.0 (age/sex lookups)
GROWTH_RECORDS	Raw + calculated growth measurements	2.0	2.0 (history/trend charts)
GROWTH_REFERENCE_DATA	LMS parameters by sex/age/metric	— (preloaded reference)	2.0 (percentile/SDS calc)
PUBERTY_SCREENINGS	Questionnaire responses + result	3.0	3.0 (history)
BONE_AGE_PREDICTIONS	Predicted bone age + metadata	4.0	4.0 (history)
Object storage (X-ray images)	Raw image binaries	4.0 (upload step)	4.0 (forwarded to AI service)
ARTICLES	Knowledge/education content	— (content-managed)	5.0

Note: ARTICLES is not explicitly listed among the database schema items in TOR Section 3.3, which names five core stores (user accounts, child profiles, growth records, screening data, and bone age prediction records) plus the growth-percentile reference tables. It has been added here to support FR-20/FR-21 and should be confirmed with the Client Representative as either a database table or static front-end content.

6. Summary

Taken together, the backend for GrowTH is organized around five request-handling modules — account and profile management, growth tracking, puberty screening, AI-assisted bone age prediction, and knowledge content — all of which share the same account-scoped access rule and all of which write to a small, well-defined set of data stores. The only module that reaches outside the system entirely is bone age prediction, which hands off to an external AI inference service and is therefore the flow with the most steps and the most points where validation and error handling matter.

This narrative description, together with the earlier ERD and DFD diagrams, should give the development team enough detail to begin implementing the API endpoints, the database schema, and the integration with the AI inference service without needing to re-derive the data flow from the TOR text directly.

Data schema

1. users — บัญชีผู้ใช้

Column	PostgreSQL Type	Key	คำอธิบาย
id	UUID	PK	รหัสผู้ใช้
full_name	VARCHAR(150)		ชื่อ-นามสกุล
email	VARCHAR(255)	UK	อีเมล ใช้ login
password_hash	VARCHAR(255)		รหัสผ่านแบบ hash (Ecrypt)
phone_number	VARCHAR(30)		เบอร์โทร
terms_accepted	BOOLEAN		ยอมรับเงื่อนไขหรือยัง
terms_accepted_at	TIMESTAMPTZ		เวลาที่ยอมรับ
created_at / updated_at	TIMESTAMPTZ		เวลาสร้าง/แก้ไข

2. sessions — เซสชันการเข้าสู่ระบบ

Column	PostgreSQL Type	Key	คำอธิบาย
id	UUID	PK	รหัสเซสชัน
user_id	UUID	FK → users	เจ้าของเซสชัน
token_hash	VARCHAR(255)	UK	hash ของ token
created_at	TIMESTAMPTZ		เวลาเริ่ม session
expires_at	TIMESTAMPTZ		วันหมดอายุ
revoked_at	TIMESTAMPTZ		เวลาที่ถูก revoke (nullable)

3. children — โปรไฟล์เด็ก

Column	PostgreSQL Type	Key	คำอธิบาย
id	UUID	PK	รหัสเด็ก
user_id	UUID	FK → users, ON DELETE CASCADE	ผู้ปกครองเจ้าของโปรไฟล์

name	VARCHAR(150)		ชื่อเด็ก
sex	sex_type (ENUM: male/female)		เพศ
date_of_birth	DATE		วันเกิด
relationship	relationship_type (ENUM)	*ตัดตดออกก	ความสัมพันธ์กับผู้ใช้
created_at / updated_at	TIMESTAMPTZ		เวลาสร้าง/แก้ไข

4. growth_reference_data — ข้อมูลอ้างอิงการเจริญเติบโต

Column	PostgreSQL Type	Key	คำอธิบาย
id	UUID	PK	รหัสแถวอ้างอิง
sex	sex_type (ENUM)		เพศที่ใช้ค่านี้
age_months	INTEGER		อายุ (เดือน)
metric_type	metric_type (ENUM: height/weight/bmi)		ชนิดตัวชี้วัด
l_value / m_value / s_value	DOUBLE PRECISION		พารามิเตอร์ LMS สำหรับคำนวณ SDS/percentile

(UNIQUE: sex + age_months + metric_type)

5. growth_records — บันทึกการเจริญเติบโต

Column	PostgreSQL Type	Key	คำอธิบาย
id	UUID	PK	รหัสบันทึก
child_id	UUID	FK → children, ON DELETE CASCADE	เด็กเจ้าของข้อมูล
measurement_date	DATE		วันที่วัด
height_cm	NUMERIC(5,2)		ส่วนสูง (ซม.)
weight_kg	NUMERIC(5,2)		น้ำหนัก (กก.)
bmi	NUMERIC(5,2)		ดัชนีมวลกาย (คำนวณ)

height_percentile / height_sds	NUMERIC(5,2) / (5,3)		เปอร์เซ็นต์ไทล์และ SDS ส่วนสูง
weight_percentile / weight_sds	NUMERIC(5,2) / (5,3)		เปอร์เซ็นต์ไทล์และ SDS น้ำหนัก
bmi_percentile / bmi_sds	NUMERIC(5,2) / (5,3)		เปอร์เซ็นต์ไทล์และ SDS BMI
guidance_message	TEXT		ข้อความแนะนำแบบ plain-language
is_flagged	BOOLEAN		ผลอยู่นอกช่วงปกติหรือไม่
created_at	TIMESTAMPTZ		เวลาบันทึก

(UNIQUE: child_id + measurement_date)

6. puberty_screenings — แบบคัดกรองวัยเจริญพันธุ์

Column	PostgreSQL Type	Key	คำอธิบาย
id	UUID	PK	รหัสแบบคัดกรอง
child_id	UUID	FK → children, ON DELETE CASCADE	เด็กที่ทำแบบคัดกรอง
screening_date	DATE		วันที่ทำแบบคัดกรอง
responses	JSONB		คำตอบ (โครงสร้างต่างกันตามเพศ/อายุ — Annex D)
result_summary	TEXT		สรุปผล (screening aid ไม่ใช่การวินิจฉัย)
recommended_action	TEXT		คำแนะนำขั้นถัดไป
created_at	TIMESTAMPTZ		เวลาบันทึก

7. bone_age_predictions — ผลทำนายอายุกระดูก

(ปรับตามตาราง reconciliation — เพิ่ม *confidence_score* ที่หายไปจาก schema เดิม)

Column	PostgreSQL Type	Key	คำอธิบาย
--------	-----------------	-----	----------

id	UUID	PK	รหัสผลทำนาย
child_id	UUID	FK → children, ON DELETE CASCADE	เด็กที่ถูกทำนาย
xray_image_ref	VARCHAR(500)		path/key ไปยังไฟล์ภาพใน object storage (ไม่เก็บไฟล์จริงในนี้)
predicted_bone_age_months	NUMERIC(5,2)		อายุกระดูกที่ทำนาย (เดือน)
margin_of_error	NUMERIC(5,2)		ค่าความคลาดเคลื่อน
confidence_score	NUMERIC(3,2)		 ความมั่นใจของโมเดล (0–1) — เพิ่มตาม reconciliation
model_version	VARCHAR(50)		เวอร์ชันโมเดลที่ใช้ทำนาย
disclaimer_shown	BOOLEAN		แสดง disclaimer "ไม่ใช่การวินิจฉัย" แล้วหรือยัง
predicted_at	TIMESTAMPTZ		เวลาที่ทำนาย

8. articles — บทความให้ความรู้

Column	PostgreSQL Type	Key	คำอธิบาย
id	UUID	PK	รหัสบทความ
title	VARCHAR(255)		หัวข้อบทความ
topic	VARCHAR(100)		หมวดหมู่ (ใช้ค้นหา/กรอง)
summary	TEXT		สรุปย่อ
content	TEXT		เนื้อหาเต็ม
source_attribution	TEXT		แหล่งที่มา (ห้ามละเมิดลิขสิทธิ์ atlas ตาม TOR 11.1)
published_at	TIMESTAMPTZ		วันที่เผยแพร่

Tab 4

องค์ประกอบข้อมูลในระบบ GrowTH แบ่งได้เป็นหลายระดับความอ่อนไหว มาดูกันว่ากลุ่มครับ

สรุปข้อมูลที่ต้องเก็บ แบ่งตามระดับความปลอดภัย

● ระดับทั่วไป — ป้องกันมาตรฐาน (access control ธรรมดาทั่วไป)

ข้อมูล	อยู่ในตาราง	หมายเหตุ
ข้อมูลอ้างอิงเปอร์เซ็นต์ไทล์การเจริญเติบโต	growth_reference_data	ข้อมูลสาธารณะ (WHO/มาตรฐาน) ไม่ผูกกับบุคคล
บทความให้ความรู้	articles	เนื้อหาสาธารณะ ไม่มีข้อมูลส่วนบุคคล

● ระดับปานกลาง — ข้อมูลส่วนบุคคลทั่วไป (Personal Data ตาม PDPA)

ข้อมูล	อยู่ในตาราง	ความเสี่ยงถ้ารั่วไหล
ชื่อ, อีเมล, เบอร์โทรของผู้ปกครอง	users	ระบุตัวตนได้ (identity theft, spam)
ชื่อ, เพศ, วันเกิดของเด็ก, ความสัมพันธ์	children	ระบุตัวตนเด็กได้ — อ่อนไหวกว่าข้อมูลผู้ใหญ่ เพราะเป็นข้อมูลเด็ก (minor's data) ซึ่งกฎหมายคุ้มครองเข้มกว่าปกติ

● ระดับสูง — ต้องมีมาตรการความปลอดภัยขั้นสูงสุด (Sensitive/Special Category Data)

นี่คือกลุ่มที่คุณควรให้ความสำคัญเป็นพิเศษ เพราะเป็น **ข้อมูลสุขภาพของผู้เยาว์** ซึ่งภายใต้ PDPA ถือเป็น "ข้อมูลอ่อนไหว" (sensitive personal data) ที่กฎหมายกำหนดให้ต้องมีมาตรการป้องกันสูงกว่าข้อมูลทั่วไป:

ข้อมูล	อยู่ในตาราง	ทำไมถึงอ่อนไหวเป็นพิเศษ
รหัสผ่าน (hashed)	users.password_hash	ถ้าแฮกเกอร์หามา hash อ่อน หรือรั่วไหลพร้อม salt จะกระทบทุกบัญชี
น้ำหนัก/ส่วนสูง + ผลคำนวณ SDS	growth_records	เป็น ข้อมูลสุขภาพ ของเด็กโดยตรง
คำตอบแบบคัดกรองวัยเจริญพันธุ์	puberty_screenings.responses	อ่อนไหวที่สุดกลุ่มหนึ่ง — เกี่ยวกับการพัฒนาทางเพศของผู้เยาว์ ถ้ารั่วไหลอาจสร้างความอับอาย/เสี่ยงต่อการถูกล่วงละเมิดข้อมูล
ไฟล์ภาพ X-ray	object storage (bone_age_predictions.xray_image_ref ชี้ไป)	เป็นทั้ง ข้อมูลชีวภาพ (biometric-adjacent) และภาพทางการแพทย์ของเด็ก

ผลทำนายอายุ `bone_age_predictions`
กระดูก

ข้อมูลสุขภาพที่ผูกกับตัวเด็กโดยตรง

Session token `sessions.token_hash` + server secret
/ JWT signing
key

ถ้ารั่วไหล = สวมรอยเข้าบัญชีได้ทันที

มาตรการความปลอดภัยที่ควรใช้กับกลุ่ม

1. รหัสผ่าน

- ใช้ bcrypt หรือ argon2 (มี salt ในตัว) — ห้าม MD5/SHA เปลือยๆ
- ไม่มีทางถอดรหัสกลับได้แม้แต่ทีมพัฒนาเอง

2. ข้อมูลสุขภาพ (`growth_records`, `puberty_screenings`, `bone_age_predictions`)

- Encryption at rest ระดับฐานข้อมูล (Postgres รองรับผ่าน disk encryption ของผู้ให้บริการ เช่น Supabase/Neon เข้ารหัสให้อัดโนมัติ)
- Encryption in transit — บังคับ HTTPS/TLS ทุก endpoint ไม่มีข้อยกเว้น
- Row-level access control ผ่าน FR-24 (`child_id` ต้องผูกกับ `user_id` ผู้ร้องขอเท่านั้น) — บังคับใช้ทุก query ไม่ใช่แค่ที่ front end
- Audit log ว่าใครเข้าถึงข้อมูลเด็กคนไหนเมื่อไหร่ (เพื่อ Client Representative/ตรวจสอบภายหลัง)

3. ไฟล์ภาพ X-ray

- เก็บใน object storage แยกจาก DB หลัก (ตามที่ออกแบบไว้แล้ว)
- ห้าม ตั้งเป็น public bucket — ต้องเป็น private bucket + generate signed URL อายุสั้น (เช่น 5-10 นาที) ตอนต้องแสดงผลเท่านั้น
- ไม่ log path เต็มของไฟล์ลงใน log ทั่วไป

4. Session / JWT

- เก็บ signing secret ใน environment variable หรือ secrets manager ห้าม hardcode ในโค้ด
- เก็บใน DB แค่ `token_hash` ไม่เก็บ token ดิบ (เหมือนหลักการเดียวกับ password)
- ตั้ง expiry สั้นและมี revoke mechanism (มีอยู่แล้วใน schema — `revoked_at`)

5. Backup

- Backup ต้อง encrypted เช่นกัน ไม่ใช่แค่ตัว production DB
- จำกัดคนที่เข้าถึง backup ได้

หลักการ PDPA ที่ TOR อ้างถึง (Section 6.2) ที่ควรยึดคู่กับตารางนี้

- **Data minimization** — เก็บเฉพาะที่จำเป็นจริงๆ (เช่น ไม่ต้องเก็บที่อยู่เต็มถ้าไม่ได้ใช้)
- **Purpose limitation** — ข้อมูลสุขภาพเด็กใช้เพื่อ screening เท่านั้น ห้ามเอาไปทำการตลาด/ขายต่อ
- **Right to deletion** — ต้องมี mechanism ลบข้อมูลเด็กทั้งหมด (รวม X-ray ในอัลบั้มเจ็ดสต่อเรจ) เมื่อผู้ปกครองร้องขอ

ถ้าต้องการ ผมสามารถเขียนตารางนี้ (พร้อมมาตรการ) ใส่เป็นภาคผนวกในเอกสาร Word ที่ทำไว้ก่อนหน้านี้ หรือแยกเป็นเอกสาร "Data Security & Privacy Policy" เดี่ยวๆ ก็ได้ครับ

AI+Backend

GROWTH · PROJECT BETA

GrowTH Project

Consolidated Technical Notes

Merged from four working notes: Web Application Overview, Backend Requirements, AI Bone Age Model, and the AI ↔ Frontend ↔ Backend Integration Flow

Prepared for: Project Beta — Digital Media Engineering Program, Khon Kaen University

Source notes merged: GrowTH_Web_Application.md; Backend_for_GrowTH.md;

AI-assisted_bone_age_prediction_from_hand_or_wrist_X-rays.md; GrowTH_AI_Frontend_Flow.md

Companion documents: GrowTH_Backend_Technical_Description.docx;

GrowTH_FastAPI_Postgres_Implementation_Guide.docx; growth_schema.sql

Document status: Working consolidation — see Section 5 for naming conflicts that need team alignment

Table of Contents

1. Project Overview
2. Backend Requirements Summary
3. AI Bone Age Prediction Model
4. End-to-End Integration Flow (Frontend ↔ Backend ↔ AI)
5. Naming and Schema Reconciliation Notes
6. Deployment and Testing Checklists
7. Summary

1. Project Overview

1.1 Description and Core Features

GrowTH is a responsive web application that helps parents monitor children's growth and development. It combines four features into a single product:

- Growth tracking
- Puberty development screening
- AI-assisted bone age prediction from hand or wrist X-rays (backed by the GrowTH backend)
- Educational content about child growth and puberty

1.2 Client Representative Responsibilities

The client is a pediatric healthcare stakeholder. Per the TOR, the Client Representative is responsible for:

- Approving scope and design
- Providing medical feedback
- Reviewing growth standards
- Reviewing the puberty questionnaire
- Reviewing explanations for AI results
- Coordinating the physician interview

1.3 Target User Personas

Primary Persona: Parents and Caregivers

Typically people who have one or multiple children, worry about growth, want to know whether development is normal, and are not medical experts. They need:

- Simple language
- Easy-to-read charts
- Clear explanations
- No medical jargon

Secondary Users: Doctors (indirectly)

Doctors don't actively use the system, but may review a child's growth history, bone age results, and puberty screening history when a parent brings the app's output to a consultation.

2. Backend Requirements Summary

Condensed from the Backend_for_GrowTH note — this matches what was already built out in detail in the companion Backend Technical Description and growth_schema.sql:

- Design a relational database schema covering user accounts, child profiles, growth measurement records, puberty screening responses/results, bone age prediction records (image reference + result), and reference tables for growth percentile data.
- Build a documented REST-ish API covering auth, profile management, growth data CRUD, screening submission/retrieval, and bone age prediction request/response.
- Implement authentication and session management with hashed/salted passwords.
- Enforce access control — a parent account can only see data for children linked to it (FR-24).
- Implement server-side calculation logic for growth percentiles/SDS and compiling puberty screening results.
- Deploy backend and database to an accessible environment for demo/testing/grading.
- Add basic logging and error handling.
- Document data privacy handling — what's collected, stored, and what could be deleted on request — in the spirit of Thailand's PDPA.

3. AI Bone Age Prediction Model

Goal: a model that takes a hand or wrist X-ray and returns an estimated bone age, callable by the backend.

3.1 AI Technology Stack

Layer	Technology
Language	Python
Deep learning	PyTorch
CNN	Transfer learning (EfficientNet-B0)
Image processing	OpenCV / Pillow
Data handling	Pandas

API / Backend	FastAPI
Database	PostgreSQL

3.2 Development Tasks

- Acquire and prepare the RSNA Pediatric Bone Age Dataset (12,611 training and 1,425 validation images, labeled with bone age in months and sex). — **done**
- Preprocess and/or augment the dataset.
- Train a CNN for bone age regression, optionally using sex as an auxiliary input.
- Evaluate with Mean Absolute Error (MAE) in months on a held-out test set, and compare against published RSNA challenge results.
- Document model limitations — the UI-facing text must state this is a screening aid, not a diagnosis.

3.3 Exploratory Data Analysis Findings

EDA was run on the training dataset to check its structure and quality before training:

- The training set has 12,611 rows across 3 columns: id (int64), boneage (int64), male (bool). No missing values in any column.
- Descriptive statistics for boneage: mean $\approx$ 127.32 months ($\approx$ 10 years 7 months), std $\approx$ 41.18, min = 1 month, max = 228 months.

None

```
Statistic: id, boneage
count: 12611.00, 12611.00
mean: 8537.65, 127.32
std: 4108.76, 41.18
min: 1377.00, 1.00
25%: 5074.50, 96.00
50%: 8565.00, 132.00
75%: 12091.50, 156.00
max: 15610.00, 228.00
```

- The bone age histogram shows most data clustered between 130–150 months ($\approx$ 10 years 10 months to 12 years 6 months) — close to half the dataset falls in that child/pre-teen range.
- Sex distribution is roughly balanced, with male images slightly outnumbering female images by around 1,000 images.
- Image sizes vary across the dataset, so a resizing step is required in preprocessing; no broken or missing images were found.

Conclusion: the dataset is clean (no nulls, no corrupt images) but has varying image resolutions that must be normalized during preprocessing, and a sex imbalance small enough not to require special rebalancing.

3.4 Model Architecture

EfficientNet-B0 backbone with a regression head, optionally taking sex as an auxiliary input:

Input Image (224x224x3)

| EfficientNet-B0 Backbone (pretrained on ImageNet)

- | Global Average Pooling (1280 features)
- | Concat with Sex Auxiliary Input (0 or 1)
- | Dense Layer (1280 + 1) -> 256 neurons + ReLU
- | Dense Layer (256) -> 128 neurons + ReLU
- | Output Layer (128) -> 1 neuron (bone age in months)

Inference preprocessing: resize to 224×224, convert to a tensor in the 0–1 range, normalize using ImageNet mean [0.485, 0.456, 0.406] and std [0.229, 0.224, 0.225], then add a batch dimension before the forward pass (model.eval() with torch.no_grad()).

4. End-to-End Integration Flow (Frontend ↔ Backend ↔ AI)

This section merges the detailed AI ↔ Frontend ↔ Backend Flow note with the backend design already in place.

4.1 High-Level Architecture

FRONTEND (React)
| HTTP POST: multipart/form-data (X-ray image file)
v
BACKEND API (FastAPI)
| function call: predict_bone_age() (image tensor)
v
AI MODEL (PyTorch)
| return: predicted_bone_age, confidence
v
BACKEND API
| save to PostgreSQL, HTTP 200 response
v
FRONTEND (React) -- displays result to user

4.2 Frontend Upload Flow (React)

Component: XrayUploadForm.jsx — lets a parent/caregiver upload a hand-wrist X-ray for a selected child.

1. Parent navigates to the Bone Age section and selects the active child (already set up via child profile management).
2. Parent clicks “Upload X-Ray Image” and XrayUploadForm opens.
3. Parent selects an image file; client-side validation checks format (JPEG/PNG only) and size (max 10MB), showing an error immediately if invalid.
4. Parent clicks “Upload & Predict Bone Age”; a loading spinner appears.
5. The browser sends a POST request to the backend with the file and child_id.
6. The backend processes the request (roughly 5–30 seconds depending on the model).
7. BoneAgeResult displays the plain-language bone age, margin of error, and screening disclaimer, with an optional expandable “Technical Details” section.
8. The parent can upload another X-ray or navigate elsewhere; the result is saved and shown in BoneAgeHistoryList, linked to the child.

Front-end files involved: src/components/BoneAge/XrayUploadForm.jsx (form logic), BoneAgeResult.jsx (result display), BoneAgeHistoryList.jsx (history view). XrayUploadForm keeps local state for file, loading, error, and result.

4.3 API Contract

Endpoint: POST /api/bone-age/predict — Content-Type: multipart/form-data.

Form Data:

- file: <binary image file (JPEG or PNG)>
- child_id: "uuid-12345"
- sex: "M" or "F" (optional, helps the model if available)

Headers: Authorization: Bearer {authToken}

Success response (200 OK):

JSON

```
{
  "prediction_id": "pred-abc12345678",
  "predicted_bone_age_months": 142,
  "margin_of_error_months": 6,
  "model_accuracy_mae": "±6 months",
  "screening_note": "This prediction supports screening and is not a clinical diagnosis.",
  "image_path": "/uploads/child-12345_2026-07-15_pred-abc12.jpg",
  "created_at": "2026-07-15T10:30:45Z",
  "confidence_score": 0.87
}
```

Error responses:

Code	HTTP Status	Meaning	User-facing message
INVALID_FILE	400	File not JPEG/PNG	Invalid file format. Please upload a JPEG or PNG image.
FILE_TOO_LARGE	400	File > 10MB	File too large. Maximum size: 10MB.
INVALID_IMAGE	400	Corrupted image	Invalid image file. Could not read image.
UNAUTHORIZED	401	No/invalid token	Please log in to upload images.
NOT_FOUND	404	Child ID doesn't exist	Child profile not found.
INFERENCE_ERROR	500	Model failed	An error occurred during prediction. Please try again.
DB_ERROR	500	Database error	Server error. Please try again later.

4.4 Backend Processing Steps

1. Validate file type — check the Content-Type header is image/jpeg or image/png; return 400 if not.
2. Validate file size — read the file into memory and check it is $\leq$ 10MB; return 400 if not.
3. Verify image integrity — open with PIL and call image.verify() to check for corruption; return 400 on failure.

4. Save the image to disk (or object storage) with a unique filename such as {child_id}_{timestamp}_{prediction_id}.jpg, converting to RGB if needed.
5. Call the AI model — build the image tensor and call predict_bone_age(image_path, model), which returns (predicted_bone_age, confidence).
6. Store the result — insert a row into bone_age_predictions with prediction_id, child_id, image_path, predicted_bone_age_months, confidence_score, and created_at, linked via the child_id foreign key.
7. Build and return the response — compile bone age, margin of error, disclaimer text, confidence, image path, and timestamp into the JSON response and return 200 OK.

4.5 Timeline and Error Scenarios

Time	Event
T=0s	Parent selects X-ray file in browser
T=0.2s	Client-side validation passes, user clicks Upload
T=0.5s	Backend receives request, validates file
T=1s	Image saved to disk
T=2s	Model inference runs (forward pass)
T=2.5s	Prediction returned (e.g. 142 months, confidence 0.87)
T=3s	Backend writes record to PostgreSQL
T=3.5s	HTTP 200 response sent back to frontend
T=4.5s	BoneAgeResult component renders and displays

Three scenarios are worth designing for explicitly: (1) an invalid file format is caught client-side and never reaches the backend; (2) a valid upload flows straight through to a successful, saved prediction; (3) the AI model itself fails (for example, a CUDA out-of-memory error) — the backend must catch this, return a clean 500 error, and the front end shows a generic “please try again” message without corrupting any saved data.

5. Naming and Schema Reconciliation Notes

The AI ↔ Frontend ↔ Backend Flow note was written somewhat independently from growth_schema.sql and uses different names and types for the same concepts in a few places. These should be resolved by the team before implementation starts, so the frontend, backend, and database all agree on one contract.

Reconciliation Table

Concept	growth_schema.sql	AI_Frontend_Flow.md	Recommendation
Children table name	children	child_profiles	Standardize on children
Prediction PK	id: UUID	prediction_id: VARCHAR(50)	Keep UUID for consistency

Image ref field	xray_image_ref	image_path	Pick one name
Margin of error	margin_of_error	margin_of_error_months	Adopt margin_of_error_months
Confidence score	not present	confidence_score	Add confidence_score: FLOAT
Disclaimer handling	disclaimer_shown	screening_note string	Keep disclaimer_shown audit flag
Timestamp	predicted_at: TIMESTAMPTZ	created_at: TIMESTAMP	Keep predicted_at

Action item: before the AI/backend integration milestone, confirm this table with both the AI team and backend team so the API contract in Section 4.3 and the database schema use identical field names end to end.

6. Deployment and Testing Checklists

Frontend (Vercel)

- XrayUploadForm component complete
- BoneAgeResult component styled
- Error handling and validation working
- Responsive design (mobile + desktop)
- API endpoint URL correct in fetch()
- Loading spinner visible during upload

Backend (FastAPI server)

- Model loaded on startup
- File upload endpoint implemented
- Database insert working
- Error responses formatted correctly
- CORS headers allow the frontend domain
- Logging implemented for debugging
- Server deployed to a stable URL

AI Model

- Model file (.pth) exported
- Inference function tested locally
- Input/output shapes verified
- Preprocessing pipeline matches training
- MAE evaluation documented

Database (PostgreSQL)

- Table created with the agreed schema (see Section 5)
- Indexes created on child_id and predicted_at
- Foreign key constraint enforced against children
- Backup strategy in place

Testing

- Unit: file validation, image integrity check, database insert, error responses, inference function with a dummy image
- Integration: full flow upload → inference → save → return; concurrent uploads; history query
- Frontend: file input validation, error message display, result display, responsive layout on mobile

7. Summary

These four notes cover four different layers of the same system: what GrowTH is and who it's for (Section 1), what the backend must provide (Section 2), how the AI model is being built and what the data looks like (Section 3), and exactly how a request travels from a parent's browser through the backend to the AI model and back (Section 4). The one piece of unfinished work is Section 5 — aligning field names between the AI team's note and the existing database schema — which should happen before any of the bone age endpoints are implemented, so nobody has to rename fields mid-build.